

Randomized Data Structures: Hash Tables

CS1200: Introduction to Algorithms and their Limitations

Lecture 7

Fall 2026

Harvard SEAS

Put away your devices

Recap: randomized algorithms

Recap: randomized algorithms

	Las Vegas	Monte Carlo
Answer	always correct	for every x , $\Pr[B(x) \text{ correct}] \geq 1 - p$
Running time	bound on the <i>expectation</i> : $T_{\text{expected}}(n) = \max_{x: \text{size}(x) \leq n} \mathbb{E}[T_B(x)]$	bound on <i>every</i> run
Example	QuickSelect, $O(n)$	EstimateBySampling, $O(1/\epsilon^2)$

Both guarantees hold for **every input**.

Definition (Ch. 11). A data structure (Preprocess, EvalQ, EvalU) is **randomized** if Preprocess, EvalQ, or EvalU uses $\text{random}(\cdot)$.

- **Las Vegas:** every answer is correct. The input to a data structure is a sequence of operations (inserts, deletes, queries). The time of any one operation in the sequence is a random variable; we bound its *expectation*, worst case over the input sequence.
- **Monte Carlo:** fixed running times; an answer may be wrong with small probability.

The Dictionary problem

The Dictionary abstract data type

Abstract Data Type (Dynamic) Dictionary

Given: a universe size U ; every key is $K \in [U]$.

State: a multiset of key-value pairs (K, V) .

Updates: $\text{insert}(K, V)$; $\text{delete}(K)$: remove *one* pair with key K , if any.

Queries: $\text{search}(K)$: return *one* pair with key K , or $\perp$.

What data structure have we already seen that solves this abstract data type and how fast?

A balanced BST: insert, delete, search each $O(\log n)$.

A BST also maintains *order*: min, max, predecessor, successor, select, rank. DICTONARY does not require this.

Can we do better?

Theorem 11.1

Given a positive integer m as a parameter, DICTIONARY has a Las Vegas randomized data structure, HashTable, in which insert, delete, and search take **expected** time

$$O\left(1 + \frac{n}{m}\right),$$

where n is the number of pairs currently stored. Preprocess: $O(m)$ space, $O(1)$ time.

With $m = 2n$, the cost per operation is $O(1 + \frac{1}{2}) = O(1)$

Why “1+”? Even an empty table costs $O(1)$.

**Warm-up: when the universe is
fixed**

In groups: a dictionary when the universe is fixed

Keys come from a fixed universe $[U]$.

1. Design a deterministic data structure for `DICTIONARY` with $O(1)$ insert, delete, and search. What does Preprocess build, and what does each operation do?
2. It is $O(1)$ everywhere and completely deterministic. What is the catch?

	algorithm	cost
Preprocess	array A of U empty lists	$O(U)$ space, $O(1)$ time
$\text{insert}(K, V)$	add (K, V) at the head of $A[K]$	$O(1)$
$\text{delete}(K)$	remove the head of $A[K]$, if any	$O(1)$
$\text{search}(K)$	return the head of $A[K]$, else $\perp$	$O(1)$



(textbook Figure 11.1, $U = 6$)

The catch

Proposition 11.2. insert, delete, search in $O(1)$; Preprocess: $O(U)$ space, $O(1)$ time.

keys	U
nine-digit ID numbers	10^9
one machine word	$2^{64} \approx 1.8 \times 10^{19}$

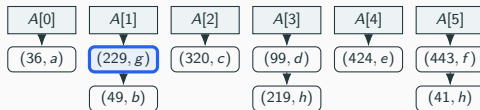
Store $n = 1,000$ pairs with nine-digit keys: an array of a billion cells, almost all empty.

How can we fix this? allocate $m \approx n$ cells instead of U . Send key K to cell $h(K) = K \bmod m$.

Shrinking the table

$$h(K) = K \bmod m$$

Array of length m , so Preprocess uses $O(m)$ space. Pair (K, V) goes to the list at $A[K \bmod m]$.
($U = 601$, $m = 6$)



- **What goes wrong?** Distinct keys share a cell — a **collision**. $\text{search}(49)$ finds $(229, g)$ at the head.
- **How do we fix it?** Walk the list until the key matches.
- **What is the runtime of the fix?**
 - search and delete now cost the length of the walk.
 - If the keys spread evenly over $m = \Theta(n)$ cells, the lists have length $O(1)$. *If.*

In groups: break it

$m = 6$, keys from $[601]$, $h(K) = K \bmod 6$. The table will hold n pairs.

1. You are the adversary and you know h . Choose the n keys to insert so that search is as slow as possible. How slow is it?
 2. Would a different *fixed* function $h : [U] \rightarrow [m]$ help?
-
1. Any n keys $\equiv r \pmod{6}$, e.g. 2, 8, 14, $\dots$: one list of length n , the other five empty. A search for a key at the end of the list (or absent) walks all n pairs: $\Theta(n)$, worse than a BST.
 2. No. *By the pigeonhole principle*: h maps the U possible keys into m cells, so some cell c receives at least U/m of them. Whenever U is large enough that $U \geq nm$, the adversary can choose n keys that all map to c , so the list at $A[c]$ has length n .

A **fixed** function has a worst-case input, whatever the function. Worst case over inputs, this table is no better than a list.

Choose the function at random

Randomness to the rescue

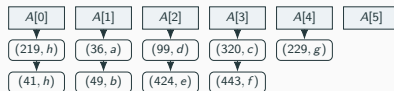
- **Deterministic h :** the adversary knows h (it is part of the algorithm), and pigeonhole proves there is a bad key set for every h .
- **Randomized h :** the adversary knows the **family** of h s but not which one will be chosen. Every key set is fine for most choices of h . (Lemma 11.3).

The family (Equation 11.1)

Fix a prime p with $U \leq p < 2U$. In Preprocess, draw $a = 1 + \text{random}(p - 1)$ and $b = \text{random}(p)$; use $h_{a,b}(K) = ((aK + b) \bmod p) \bmod m$.

Same nine keys, $p = U = 601$, $m = 6$, and the draw $a = 13$, $b = 7$:

$$h_{13,7}(36) = (475 \bmod 601) \bmod 6 = 1$$



Lemma 11.3: what we need from a hash family

A set R of labels and, for each $\ell \in R$, a function $h_\ell : [U] \rightarrow [m]$, such that:

1. **Compact description:** $|R| = O(U^2)$, so $\ell = \text{random}(R)$ costs $O(1)$ and fits in $O(1)$ words.
2. **Efficient evaluation:** $h_\ell(K)$ takes $O(1)$ time.
3. **Low collision probability:** for every $K \neq K'$ in $[U]$,

$$\Pr_{\ell} [h_{\ell}(K) = h_{\ell}(K')] = O\left(\frac{1}{m}\right) \quad (\ell \text{ uniform in } R; K, K' \text{ fixed first}).$$

For $h_{a,b}$: $R = \{1, \dots, p-1\} \times \{0, \dots, p-1\}$, $|R| < 4U^2$; three arithmetic operations; collision probability $\leq 1/m$ (proof omitted).

Why not a truly random function $h : [U] \rightarrow [m]$? Collisions exactly $1/m$, but writing h down takes $U \log m$ bits and U random choices — linear in U in both space and time.

HashTable (proof of Theorem 11.1)

- Preprocess: allocate an array A of m empty lists;
Set $\ell = \text{random}(R)$.
- $\text{insert}(K, V)$: add (K, V) at the head of $A[h_\ell(K)]$.
- $\text{delete}(K)$: walk the list at $A[h_\ell(K)]$; remove the first pair with key K .
- $\text{search}(K)$: walk the list at $A[h_\ell(K)]$; return the first pair with key K , else $\perp$.

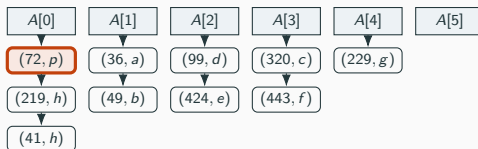
Example. $U = p = 601$, $m = 6$.

Preprocess drew $\ell = (a, b) = (13, 7)$.

$\text{insert}(72, p)$:

$h_{13,7}(72) = (943 \bmod 601) \bmod 6 = 342 \bmod 6 = 0$.

Insert $(72, p)$ at the head of $A[0]$.



Correctness: every stored pair sits in the list at its own hash, and the walk checks keys. No wrong answers, ever
 $\Rightarrow$ Las Vegas.

Expected runtime

Runtime of HashTable operations

Preprocess	$O(m)$ space for A ; $O(1)$ time to draw ℓ
insert(K, V)	$O(1)$: one hash, one head insertion
delete(K), search(K)	$O(1)$ + the length of the walk along $A[h_\ell(K)]$

What determines how long the walk is? How many stored keys share K 's cell — which depends on the keys $K_0, \dots, K_{n-1}$ and on ℓ .

What makes this an “expected” time calculation? ℓ was drawn at random; the keys are the adversary's, not random. So “expected” means $\mathbb{E}_\ell$, for every fixed set of keys.

- The walk passes at most one pair with key K , plus every stored pair with a *different* key in K 's cell: $\text{walk}_\ell(K) \leq 1 + \#\{i : K_i \neq K \text{ and } h_\ell(K_i) = h_\ell(K)\}$.
- **Linearity of expectation:** the expected size of that set is $\sum_{i: K_i \neq K} \Pr_\ell [h_\ell(K_i) = h_\ell(K)]$.
- **Low collision probability** (property 3 of our family): each term is at most $1/m$.
- So $\mathbb{E}_\ell[\text{walk}_\ell(K)] \leq 1 + n/m$: expected time $O(1 + n/m)$. ■ Theorem 11.1

Choosing m

Load: $\alpha = n/m$. Search and delete cost $O(1 + \alpha)$. **How should we choose m ?**

- Time: $O(1 + n/m)$ is $O(1)$ whenever $m = \Omega(n)$.
- Space: $O(m)$, so we also want $m = O(n)$.
- Together: $m = \Theta(n)$ — keep the load α a constant.

The catch: n is not known when m is chosen, and it changes as pairs are inserted and deleted.

Rule: when $\alpha > 2/3$, set $m \leftarrow 2m$, draw a fresh ℓ , and re-insert all n pairs. Cost $O(m)$; the load drops to $1/3$.

How often does this happen? Only after $\Omega(m)$ inserts have raised the load from $1/3$ to $2/3$.

So the rebuilds add $O(1)$ per insert on average — *amortized* $O(1)$.

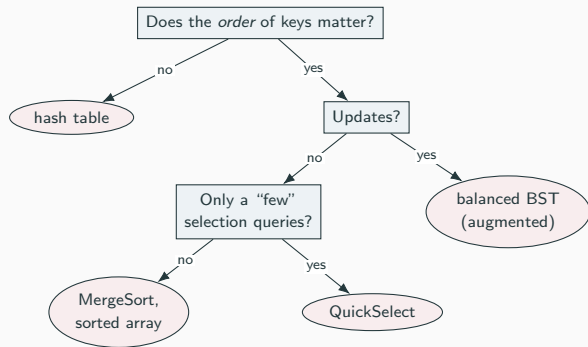
**Storing and searching: where we
are**

In groups: four ways to store and search

Consider all the data structures we have studied for storing and searching key–value pairs. **For each:**
What queries does it support? When is it an advantageous choice?

	build	search	ins / del	min, max, pred, succ	select, rank	build + 1 select
sorted array (MergeSort once)	$O(n \log n)$	$O(\log n)$	$O(n)$	$O(\log n)$	$O(1)$	$O(n \log n)$
balanced BST (size-augmented)	$O(n \log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n \log n)$
unsorted array + QuickSelect	none	$O(n)$	$O(1) / O(n)$	$O(n)$	$O(n)$	$O(n)$
hash table	$O(m)$ space	$O(1)$ exp.	$O(1)$ exp.	—	—	—

Maxwell's flowchart (Figure 11.3)



How few is "few"? Static data, q selection queries.

sort, then index $O(n \log n + q)$

$q \times$ QuickSelect $O(qn)$

QuickSelect wins when $q \ll \log n$; sorting wins when $q \gg \log n$.

In between: measure.